Web Engineering & Development (SWE 363)

Back-End Development Fundamentals

In today's lecture:

- Full-stack development
- A first look at Express

Reference:

- Zybook: 6.1, 6.4 (intro only)

Activity: Think-Pair-Share (2 min)

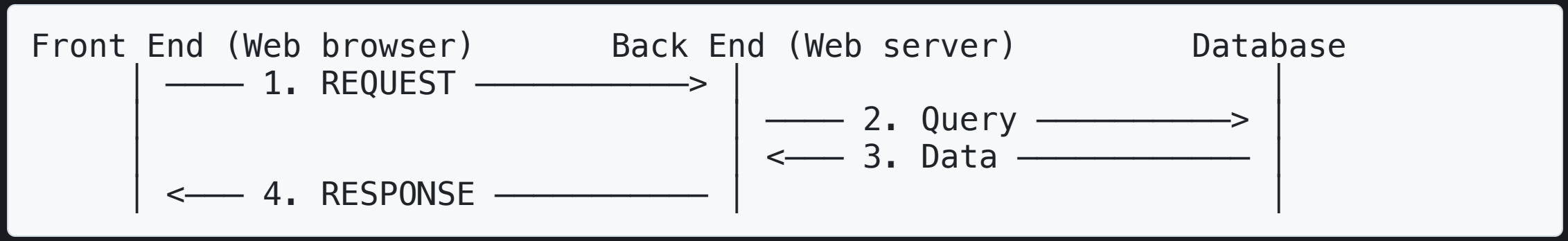
You log in to the KFUPM portal and see your grades.

1. What happened between clicking Login and seeing your grades?
2. Which parts ran on your laptop? Which ran somewhere else?
3. Where are the grades actually stored?

6.1 Full-stack Development

What is Full-Stack Development?

- **Front end** → runs in the **web browser** (HTML, CSS, JavaScript)
- **Back end** → runs on the **web server** (PHP, Node.js, Python)
- **Database** → stores the application data
- **Full-stack development** → developing **both** ends



Activity: Front end, back end, or database? (hands up)

1. An "Add to cart" button styled with CSS
2. Code that fetches and modifies product information
3. The table that stores every product
4. A React component that shows a product card
5. A Python program that generates a personalized webpage
6. A layout that works on different screen sizes

Front-End Development

- Focuses on the **user interface (UI)**
- Must work on **different devices and screen sizes**
- Must be **accessible** (e.g., screen readers)

Library = functions for a related set of tasks

Framework = a suite of libraries, a more complete platform

Name	Type
Bootstrap, Tailwind	CSS framework
React	JavaScript library
Vue.js, Angular	JavaScript framework

Back-End Development: Two Approaches

Dynamic Webpage

- Page **generated on the web server** when requested
- Personalized to the user

Web API

- Functions invoked using **HTTP**
- The front end uses **JavaScript** to call them

Activity: Dynamic webpage or web API?

1. Becky is shown a webpage that lists the items in her shopping cart.
2. A retailer's front end uses JavaScript to request all sale items, then displays them.
3. A PHP program generates a profile page for the logged-in user.

Server-Side Platforms

Platform	Language	Popular Frameworks
ASP.NET	C#	Blazor, MVC
Java	Java	Spring Boot
Node.js	**JavaScript**	**Express**, NestJS
PHP	PHP	Laravel, Symfony
Python	Python	Django, Flask, FastAPI

In this course: Node.js + Express

Databases

Relational (SQL)

- Data in **tables**
- MySQL, PostgreSQL

```sql
SELECT * FROM product
WHERE price < 900;
```

Document (NoSQL)

- Data in **JSON** format
- **MongoDB**

```
db.products.find(
  { price: { $lt: 900 } }
);
```

Activity: Predict

id	title	price
123	Laptop	350.99
445	Smart TV	1199.99
876	Tablet	499.99

1. Which products does `SELECT * FROM product WHERE price < 900;` return?
2. Adding a fourth product adds a new table, row, or column?
3. True or False: the front end pulls data directly from the database.

A First Look at Express

What is Express?

> **Express** is a popular **Node.js framework** for creating web applications and **web APIs**

Install it with npm:

```
npm install express
```

After installing, the package is in the project's `node_modules` directory

A Minimal Express Server

```
// server.js
const express = require("express");   // import the express module
const app = express();                 // create the Express app object

// routes go here

app.listen(3000, function() {          // start listening on port 3000
  console.log("Listening on port 3000...");
});
```

```
node server.js     # stop with Ctrl+C
```

- `express()` **creates** the app; it does not start the server
- `app.listen()` **starts** the server

Routes

> A **route** = an HTTP method + a URL path + a **callback function**:
>
> `app.method(path, callback)`

```
// Called for GET request to http://localhost:3000/hello
app.get("/hello", function(req, res) {
  res.send("<h1>Hello, Express!</h1>");
});

// Called for GET request to http://localhost:3000/course
app.get("/course", function(req, res) {
  res.json({ code: "SWE 363", title: "Web Engineering" });
});
```

- `req` / `res` → the HTTP **request** and **response**
- `res.send()` → sends HTML; `res.json()` → sends an object as **JSON**

Activity: Predict what the browser shows

```
app.get("/hello", function(req, res) {
  res.send("<h1>Hello, Express!</h1>");
});
app.get("/course", function(req, res) {
  res.json({ code: "SWE 363" });
});
app.listen(3000);
```

1. `http://localhost:3000/hello`
2. `http://localhost:3000/course`
3. `http://localhost:3000/contact`
4. `http://localhost/hello`

Activity: Put the steps in order

A. `app.listen()` starts the server on port 3000

B. The browser requests `http://localhost:3000/hello`

C. `require("express")` imports Express, and `express()` creates the app

D. `res.send()` sends the response, and the browser displays it

E. The `/hello` route's callback function runs

Key Takeaways

- **Front end** = browser, **back end** = server, **database** = stored data
- Back ends generate **dynamic webpages** or provide a **web API**
- **Express:** `express()` creates the app, `app.listen()` starts it
- **Route:** `app.method(path, callback)`
- `res.send()` → HTML, `res.json()` → JSON

30m

Demo

6.1 Back-End Fundamentals

Next Class

- Node.js & npm (6.2)
- Modules (6.3)